

Multimodal Large Model + Video Understanding

Function Description

After the program runs, input questions about the video through the terminal. The large model will first record a video, then analyze the video content with the question, and finally reply with the answer to the question. You can also first record a video of specified duration, and then ask the large model questions related to the video based on the video content.

Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

Then input questions related to visual understanding in the text_chat terminal. You can refer to the following examples:

```
Record a 20-second video
```

After the buzzer beeps once,

```
root@yahboom:~/LargeModel_ws# ros2 run text_chat text_chat
user input: Record a 20-second video
okay@, let me think for a moment... /[[INFO] [1775114578.876125937] [text_chat_node]: "action": ['record_video(20)'], "response": Okay, I'm going to record a 20-second video now!
user input: [[INFO] [1775114619.076473584] [text_chat_node]: "action": ['finishtask()'], "response": I've finished recording the 20-second video! Let me know if you'd like me to analyze it or do something else.
```

After the buzzer beeps once, the recording ends. Then, based on the video content, input the following:

```
Did you see my phone in the video I just recorded?
```

```
Did you see my phone in the video I just recorded?
okay@, let me think for a moment... /[[INFO] [1775114658.047244464] [text_chat_node]: "action": ['video_understanding()'], "response": I just watched the video you recorded, and I'm checking if your phone was in it!
user input: [[INFO] [1775114665.087899782] [text_chat_node]: "action": ['finishtask()'], "response": I just analyzed the video, and yes, I saw your phone in it! It was lying on the desk near the keyboard. Everything looks clear and visible!
```

Core Code Analysis

record_video Function

Source code path: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
#Opens another terminal, starts the Record_Video node program, and passes in the parameter
time_, which is the duration of the video recording
def record_video(self,time_):
    record_time_ = float(time_)
    cmd1 = f"ros2 run largemodel_arm Record_Video --ros-args -p time_{record_time_: .2f}"
    subprocess.Popen(
        [
            "gnome-terminal",
```

```

        "--title=recording",
        "--",
        "bash",
        "-c",
        f"{cmd1}; exec bash",
    ]
)

```

#The Record_video node program source code path is

LargeModel_ws/src/largemodel_arm/largemodel_arm/Record_Video.py. Let's mainly look at the color image callback function.

```

def get_RGBCallback(self,msg):
    if self.beep_start==False:
        self.start_time = time.time()
        self.beep_start = True
        self.Arm.Arm_Buzzer_On()
        time.sleep(0.5)
        self.Arm.Arm_Buzzer_Off()
        self.record_done = False

    if self.record_done!=True:
        #Convert image topic data to image data
        rgb_image = self.rgb_bridge.imgmsg_to_cv2(msg,'bgr8')
        if self.out is None:
            #print("-----")
            height, width, _ = rgb_image.shape
            fourcc = cv2.VideoWriter_fourcc(*'x264') # Use MP4 encoder
            #Write image frames to video file
            self.out = cv2.VideoWriter(
                self.Path_video,
                fourcc,
                15,
                (width, height)
            )
            self.out.write(rgb_image)
            cv2.imshow('Video Recording', rgb_image)
            key = cv2.waitKey(1)
            elapsed = time.time() - self.start_time
            #Check if the recording duration has exceeded; if so, stop recording
            if elapsed >= self.Time_ :
                cv2.destroyAllWindows()
                self.record_done = True
                self.out.release()
                self.Arm.Arm_Buzzer_On()
                time.sleep(0.5)
                self.Arm.Arm_Buzzer_Off()
                self.largemodel_arm_done_pub.publish(String(data='record_video_done'))

```

video_understanding Function

Source code path: LargeModel_ws/src/largemodel/largemodel/action_service.py

```
#After the model_service node receives video_handle, it will transmit the video to the
Alibaba Cloud large model platform, use the multimodal large model to analyze the video
content with the question, and finally return the result of the question
def video_understanding(self):
    self.get_logger().info(
        "Publish video handle."
    )
    msg = String(data="video_handle")
    self.video_handle_pub.publish(
        msg
    ) # Normalize, publish the video_handle topic, called by model_service to invoke the
    large model
```

Combined with Robotic Arm

We can also combine the robotic arm + video understanding to develop new玩法. For example, prepare several opaque paper cups. After the video starts, place an object under one of the opaque paper cups, then slowly change the positions of the paper cups. After the video recording ends, ask the large model which cup the object is finally under, and then have the robotic arm point to that cup. To test this function, enter in the terminal:

Let's play a game. First record a video, then guess which cup has the red block at the end of the video?

Replace the small yellow duck with the actual object,

```
root@yahboom:~/ros2 run text_chat text_chat
user input: Let's play a game. First record a video, then guess which cup has the red block at the end of the video?
okay@, let me think for a moment... /[INFO] [1775116425.267150151] [text_chat_node]: Decision making AI planning:1. Call the `record_video` function, pass in the parameter "20", and record
a 20-second video;
2. Call the `video_understanding` function to analyze the recorded 20-second video, and determine which cup has the red block hidden under it at the end of the video;
3. Call the `point_to(x1, y1, x2, y2)` function to point to the cup with the red block, where x1, y1, x2, y2 represent the outer frame coordinates of the cup with the red block at the end of
the video.
[INFO] [1775116427.486427567] [text_chat_node]: "action": ['record_video(20)'], "response": Okay, let's play this fun guessing game! I'll start by recording a 20-second video to capture all
the action.
[INFO] [1775116473.478677999] [text_chat_node]: "action": ['video_understanding()'], "response": Got it! The video has been recorded. Now, let me analyze it to find out which cup hides the r
ed block at the end.
[INFO] [1775116477.926222307] [text_chat_node]: "action": ['point_to(100, 200, 200, 300)'], "response": I've analyzed the video and found the cup with the red block hidden under it. Let me p
oint to it for you!
[INFO] [1775116505.532100693] [text_chat_node]: "action": ['finishtask()'], "response": I've successfully pointed to the cup with the red block! The game is complete, and I hope you enjoyed
it!
```

The task planning should be:

Call the `record\_video` function, pass in the parameter "20", and record a 20-second video;

Call the `video\_understanding` function to analyze the recorded 20-second video, and determine which cup has the red block hidden under it at the end of the video;

If the analysis result shows that there is indeed an object hidden under a cup, then continue to execute the following steps:

Call the `point\_to(x1, y1, x2, y2)` function to point to the cup with the red block, where x1, y1, x2, y2 represent the outer frame coordinates of the cup with the red block at the end of the video.